BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P - Performance Measure
E - Environment
A - Actuators
S - Semsors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.width
sef.height
self.agent_pos
self.walls
self.food_positions
self.opponents

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent
In self-agent, the agent's performance measure depends only on its own actions and static
elements but in multi-agent, other rational entities such as opponent is trying to maximize their on scores. Therefore, the environment is multi-agent.

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is a history of all the sensory inputs or data received by an agent from its environment, since it started running up to the current moment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Environment

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable.
The agent can see its position, The agent needs internal memory to track the state of the environment such as food positions, walls and opponent positions. Therefore, the agent is partially observable.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Environment

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is important because the goal of evaluating metrics is to observe the performance measure. Hence it should only evaluate the changes in the external environment state. If the agent's internal processing effort is considered for metric evaluation it would measures how hard the agent is "thinking," not what it is actually achieving and it would misalign agent's utility with the environment's goal.

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

If we intentionally hide the data of self.toxic_traps, the environment will still remain partial. But the partiality of the environment will be more than before.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding the smells_toxin sensor expands the agent's percept sequence by giving it direct information about harmful substances in the environment. Rationality in IS means choosing actions that maximize expected utility based on available knowledge. With toxin detection, the agent can avoid dangerous areas, protect itself, or seek safer alternatives.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic "Vacuum World Exploit" from Lecture 01 showed how an agent could maximize its performance measure in a faulty way: instead of cleaning, it maximized its performance by repeatedly turning the vacuum on and off in a clean room, exploiting metrics. This illustrated the danger of poorly designed metrics. Agents may exploit loopholes rather than achieve the intended goal of keeping the environment clean

Finalize and Lock Lab 01 Analytical Evaluation

FACULTY OF COMPUTING